

Статья Ретрансляция NTLM. Часть 1

 codeby.net/threads/retranslyatsiya-ntlm-chast-1.73776

raijin

June 6, 2020



Данная часть статьи является переводом. Оригинал вот [тут](#)

[Часть 2](#)

NTLM-relay - это техника позволяет находиться между клиентом и сервером для выполнения определенных действий на сервере при этом, выдавая себя за клиента. Техника может быть очень мощной и может использоваться для получения контроля над доменом Active Directory из контекста black box (без учетных данных). Целью этой статьи является разъяснение NTLM relay, а также представление ее пределов.

От Автора

Данная статья не является учебным пособием по проведению атаки, но она позволит читателю детально разобраться в технических особенностях данной атаки, ее ограничениях, и может стать основой для начала разработки собственных инструментов или понимания того, как работают текущие инструменты.

Кроме того, чтобы избежать путаницы, вспомним некоторые обозначения:

- **NT Hash** и **LM Hash** - хэшированные версии паролей пользователей. LM хэши полностью устарели, и не будут упоминаться в этой статье. NT хэш обычно ошибочно называют "NTLM хэш". Это определение путают с именем протокола, NTLM. Поэтому, когда мы говорим о хэше пароля пользователя, мы будем называть его **NT хэшем**.
- Таким образом, **NTLM - протокол аутентификации**. Также, протокол существует в 2 версиях. В этой статье, если версия влияет на объяснение, то будут использоваться термины NTLMv1 и NTLMv2. В противном случае термин NTLM будет использоваться для всех версий протокола.
- **NTLMv1 Hash** и **NTLMv2 Hash** будут терминами, обозначающие ответ на запрос клиента, для 1 и 2 версий протокола NTLM.

- **Net-NTLMv1** и **Net-NTLMv2** также являются терминами, объясняющие, использование NT хэша вместо NTLM хэша, чтобы отличить NTLM хэш от протокола. Поскольку мы не используем терминологию NTLM хэш, то эти определения использоваться не будут.
- **Net-NTLMv1 Hash** и **Net-NTLMv2 Hash** также являются терминами во избежание путаницы, но и это я также не будут использоваться в этой статье.

Введение

NTLM relay полагается, как следует из названия, на аутентификацию NTLM. Основы NTLM были представлены в статье [pass-the-hash](#). Вам стоит прочитать хотя бы часть о протоколе NTLM и локальной/удаленной аутентификации.

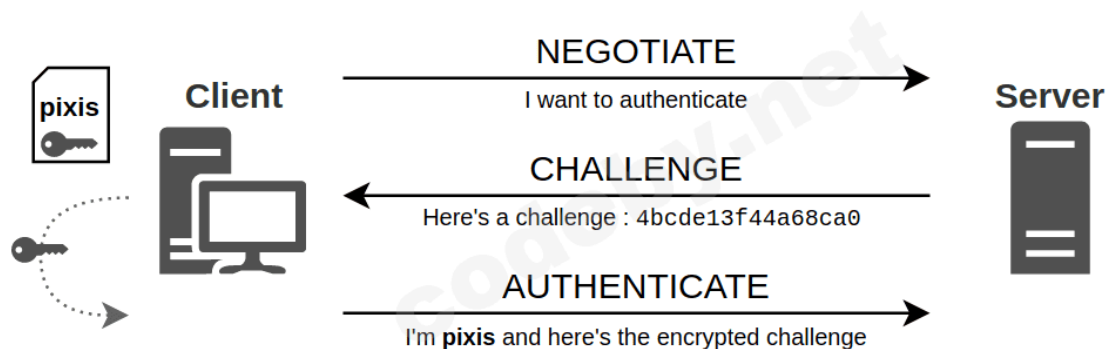
Давайте вспомним что, протокол NTLM используется для аутентификации клиента на сервере. То, что мы называем клиентом и сервером, является двумя частями обмена. Клиент - это тот, кто хочет аутентифицироваться, а сервер - это тот, кто подтверждает эту аутентификацию.



Эта аутентификация происходит в 3 этапа:

- Сначала клиент говорит серверу, что он хочет аутентифицироваться.
- Затем сервер отвечает на вызов, который представляет собой не более чем случайную последовательность символов.
- Клиент шифрует этот вызов своим секретным ключом и отправляет результат обратно на сервер. Это и есть его ответ.

Этот процесс называется вызовом/ответом.



Преимущество такого обмена заключается в том, что секрет пользователя никогда не проходит через сеть. Это известно как [Доказательство с нулевым разглашением](#).

NTLM Relay

С помощью этой информации мы можем легко представить себе следующий сценарий: Злоумышленнику удастся оказаться в положении "man-in-the-middle, человек в середине" между клиентом и сервером, и он просто передает информацию от одного к другому.

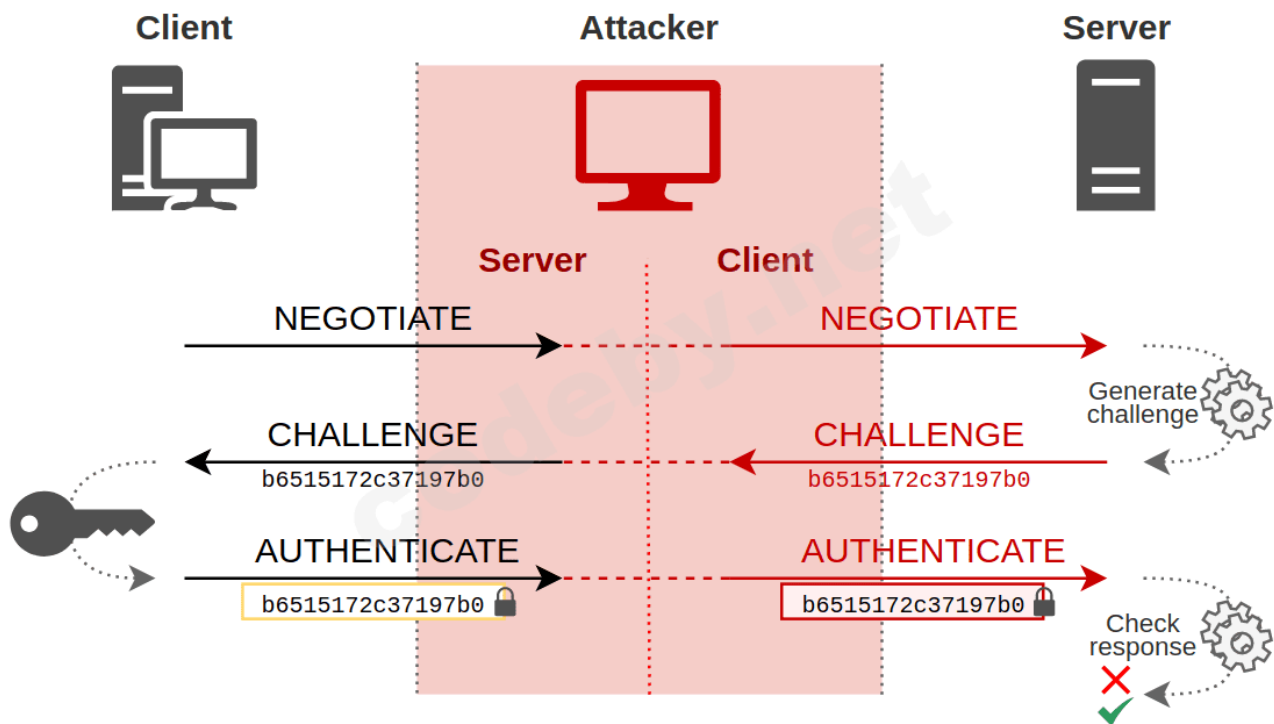
Man-in-the-middle, или человек по середине означает, что с точки зрения клиента, машина злоумышленника является сервером, на котором он хочет аутентифицироваться, а с точки зрения сервера, злоумышленник является клиентом, как и любой другой, кто хочет аутентифицироваться.

За исключением того, что злоумышленник не "просто" хочет аутентифицироваться на сервере. Он хочет сделать это, притворяясь клиентом. Однако, он не знает секрета клиента, и даже если он прослушивает разговоры, так как этот секрет никогда не передается по сети (доказательство с нулевым разглашением), злоумышленник не может извлечь никакой секрет. Так как же он работает?

Передача сообщений

Во время NTLM-аутентификации клиент может доказать серверу свою идентификацию, зашифровав своим паролем часть информации, предоставленной сервером. Таким образом, злоумышленнику остается только позволить клиенту делать свою работу и передать сообщения от клиента серверу, а ответы от сервера - клиенту.

Все, что клиент должен отправить «серверу», злоумышленник получит его и отправит сообщения обратно на реальный сервер, а все сообщения, которые сервер отправит «клиенту», злоумышленник также получит их, и он перешлет их клиенту, как есть.

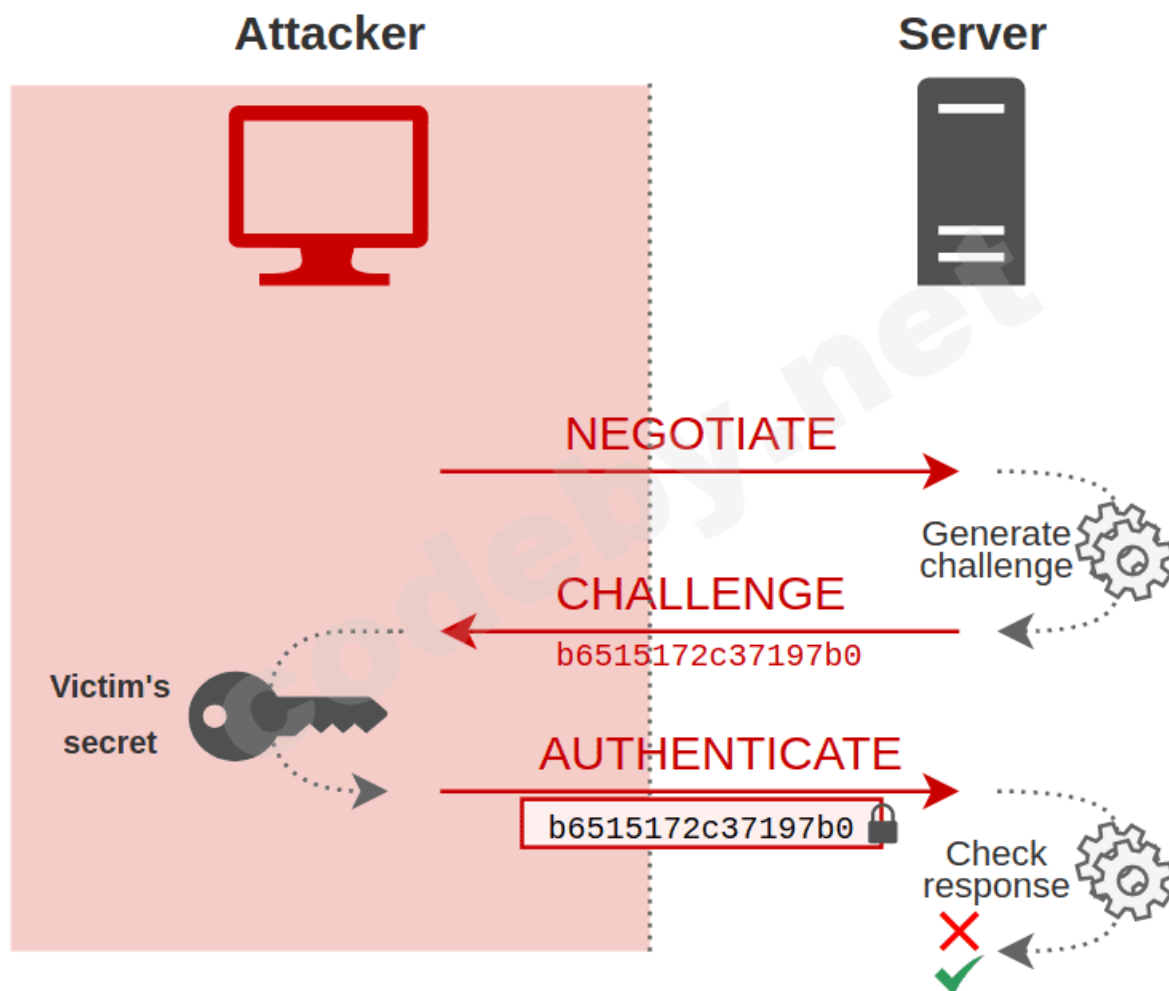


И все получается! Действительно, с точки зрения клиента, в левой части диаграммы между злоумышленником и ним происходит NTLM-аутентификация со всеми необходимыми блоками. В своем первом сообщении клиент посылает запрос, на который злоумышленник отвечает вызовом. Получив этот запрос, клиент строит свой ответ, используя свой секрет, и, наконец, посылает последнее сообщение об аутентификации, содержащее зашифрованный запрос.

Хорошо, все замечательно, но злоумышленник ничего не может сделать с этим обменом. К счастью, есть правая часть диаграммы. Действительно, с точки зрения сервера, атакующий является клиентом, как и любой другой. Он отправил первое сообщение с просьбой аутентифицироваться, и сервер ответил на него вызовом. Поскольку **злоумышленник отправил такой же запрос реальному клиенту**, реальный клиент **зашифровал этот запрос своим секретом**, и отправил **валидный ответ**. Поэтому злоумышленник может послать этот **валидный ответ** серверу.

В этом и заключается интерес к данной атаке. С точки зрения сервера, злоумышленник аутентифицировал себя, используя секрет жертвы, но прозрачным для сервера способом. Он понятия не имеет, что злоумышленник отсылал свои сообщения клиенту, чтобы заставить клиента дать ему правильные ответы.

Итак, с точки зрения сервера, вот что произошло:



В конце этих обменов атакующий проходит аутентификацию на сервере с учетными данными клиента.

Net-NTLMv1 и Net-NTLMv2

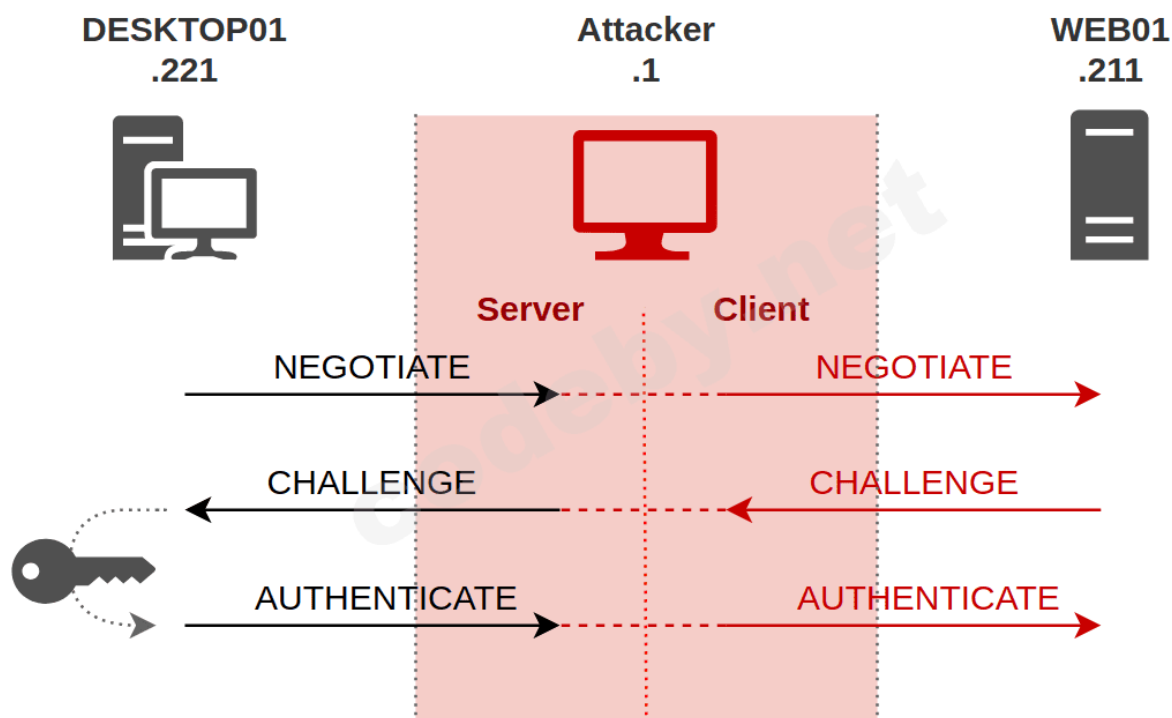
Для информации, именно этот **действительный ответ**, переданный атакующим в 3 сообщении, является зашифрованным вызовом, который обычно называется Net-NTLMv1 хэш или Net-NTLMv2 хэш. Но в этой статье она будет называться **NTLMv1 хэшем** или **NTLMv2 хэшем**, как указано в предварительном параграфе.

Если быть точным, то это не совсем зашифрованная версия вызова, а хэш, использующий секрет клиента. Например, для [NTLMv2 используется](#) функция HMAC_MD5. Этот тип хэша может быть взломан только брутфорсом. Криптография, связанная с [вычислением хэша NTLMv1](#), устарела, и NT хэш, который использовался для создания хэша, можно быстро извлечь. С другой стороны, для NTLMv2 это займет гораздо больше времени. Поэтому предпочтительно и желательно **не разрешать аутентификацию NTLMv1 в производственной сети**.

На практике

В качестве примера, я создал небольшую лабораторию с несколькими компьютерами. Есть клиент **DESKTOP01** с IP-адресом **192.168.56.221** и сервер

WEB01 с IP-адресом **192.168.56.211**. Мой хост - это злоумышленник, с IP адресом **192.168.56.1**. Таким образом, мы находимся в следующей ситуации:



Нападающий, таким образом, сумел поставить себя в положение по середине. Для этого существуют различные методики, будь то с помощью злоупотребления стандартными конфигурациями IPv6 в среде Windows, или с помощью протоколов LLMNR и NBT-NS. В любом случае, атакующий заставляет клиента думать, что он является сервером. Таким образом, когда клиент пытается аутентифицироваться, именно с ним атакующий будет выполнять эту операцию.

Инструментом, который я использовал для выполнения этой атаки, является [ntlmrelayx](#) из **impacket**. Этот инструмент подробно описан в [этой статье Agsolino](#), разработчиком **impacket** (*всемогущим*).

```
ntlmrelayx.py -t 192.168.56.221
```

Инструмент создает различные серверы, в том числе и SMB сервер для данного примера. Если он получает соединение на этом сервере, он передает его на предоставленную цель, которая в данном примере **192.168.56.221**.

С точки зрения сети, это захват обмена, при котором злоумышленник передает информацию цели.

192.168.56.221	192.168.56.1	SMB	196 Session Setup AndX Request, NTLMSSP_NEGOTIATE
192.168.56.1	192.168.56.211	SMB	214 Session Setup AndX Request, NTLMSSP_NEGOTIATE
192.168.56.211	192.168.56.1	SMB	428 Session Setup AndX Response, NTLMSSP_CHALLENGE, Error: STATUS_MORE_PROCESSING_REQUIRED
192.168.56.1	192.168.56.221	SMB	354 Session Setup AndX Response, NTLMSSP_CHALLENGE, Error: STATUS_MORE_PROCESSING_REQUIRED
192.168.56.221	192.168.56.1	SMB	638 Session Setup AndX Request, NTLMSSP_AUTH, User: ADSEC\jsnow
192.168.56.1	192.168.56.211	SMB	657 Session Setup AndX Request, NTLMSSP_AUTH, User: ADSEC\jsnow

Зеленым цветом отмечены обмены между клиентом **DESKTOP01** и атакующим, а красным - обмены между атакующим и сервером **WEB01**. Четко видны 3 NTLM

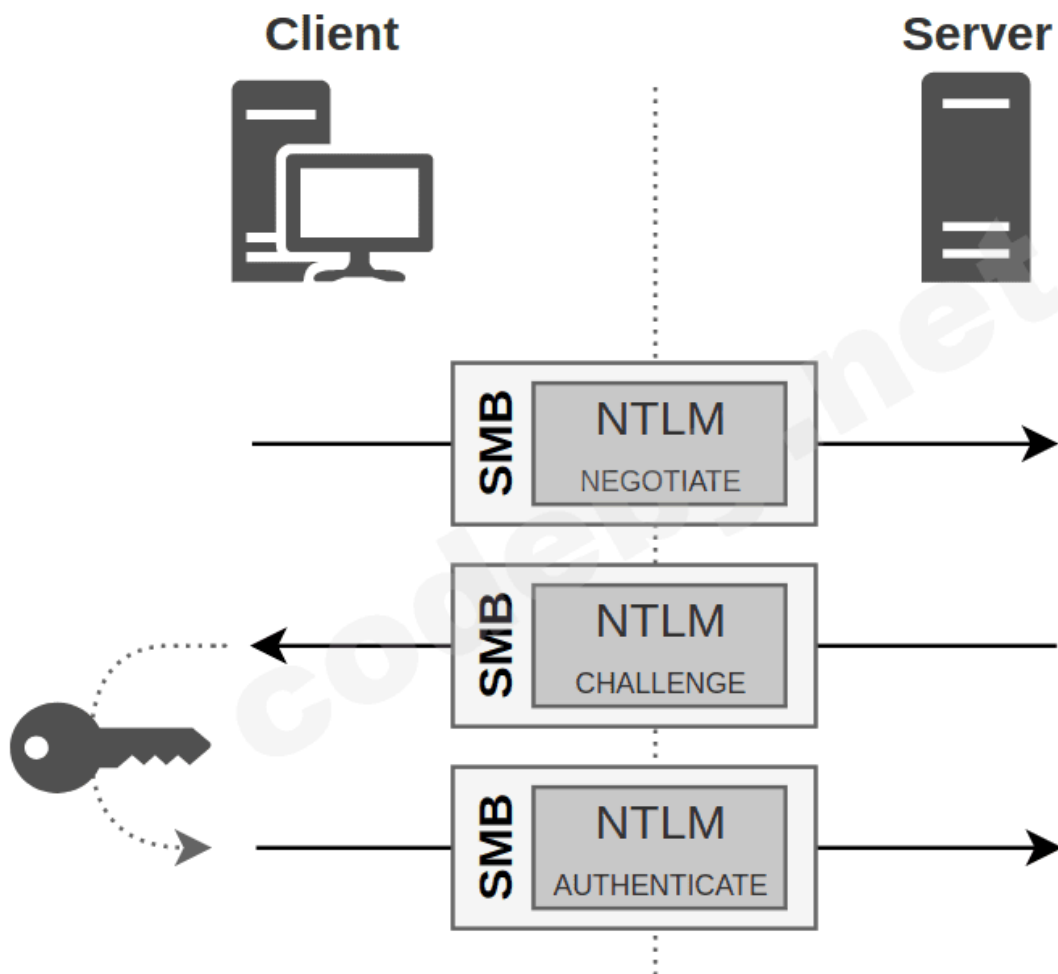
Нападающий, который получает все, что не задает вопросов. Он посылает ту же самую информацию на сервер. Поэтому он притворяется **jsnow** на **DESKTOP01** и части домена **ADSEC**, а также посылает ответ, который был вычислен клиентом, называемый **NTLM Response** на этих скриншотах. Мы называем этот ответ **NTLMv2 хэшем**.

Мы видим, что нападающий только передавал информацию. Он просто пересылал информацию с клиента на сервер и наоборот, за исключением того, что, в конце концов, сервер считает, что атакующий успешно аутентифицирован, и тогда атакующий может выполнять действия на сервере от имени **ADSEC\jsnow**.

Теперь, когда мы поняли основной принцип ретрансляции NTLM, возникает вопрос, как же конкретно мы можем выполнять действия на сервере после ретрансляции NTLM-аутентификации? Кстати, что мы понимаем под "действиями"? Что можно сделать?

- **Аутентификация**, позволяющая серверу проверить, что клиент является тем, за кого он себя выдает.
- **Сеанс**, во время которого клиент сможет выполнять действия.

Для управления этими двумя шагами используемый протокол должен быть способен инкапсулировать аутентификацию, то есть обмен NTLM сообщениями.



Конечно, если бы все протоколы интегрировали технические детали NTLM, это быстро превратилось бы в священный беспорядок. Вот почему Microsoft предоставляет интерфейс, на который можно положиться при работе с аутентификацией, а пакеты были специально разработаны для процедур с различными типами аутентификации.

SSPI и NTLMSSP

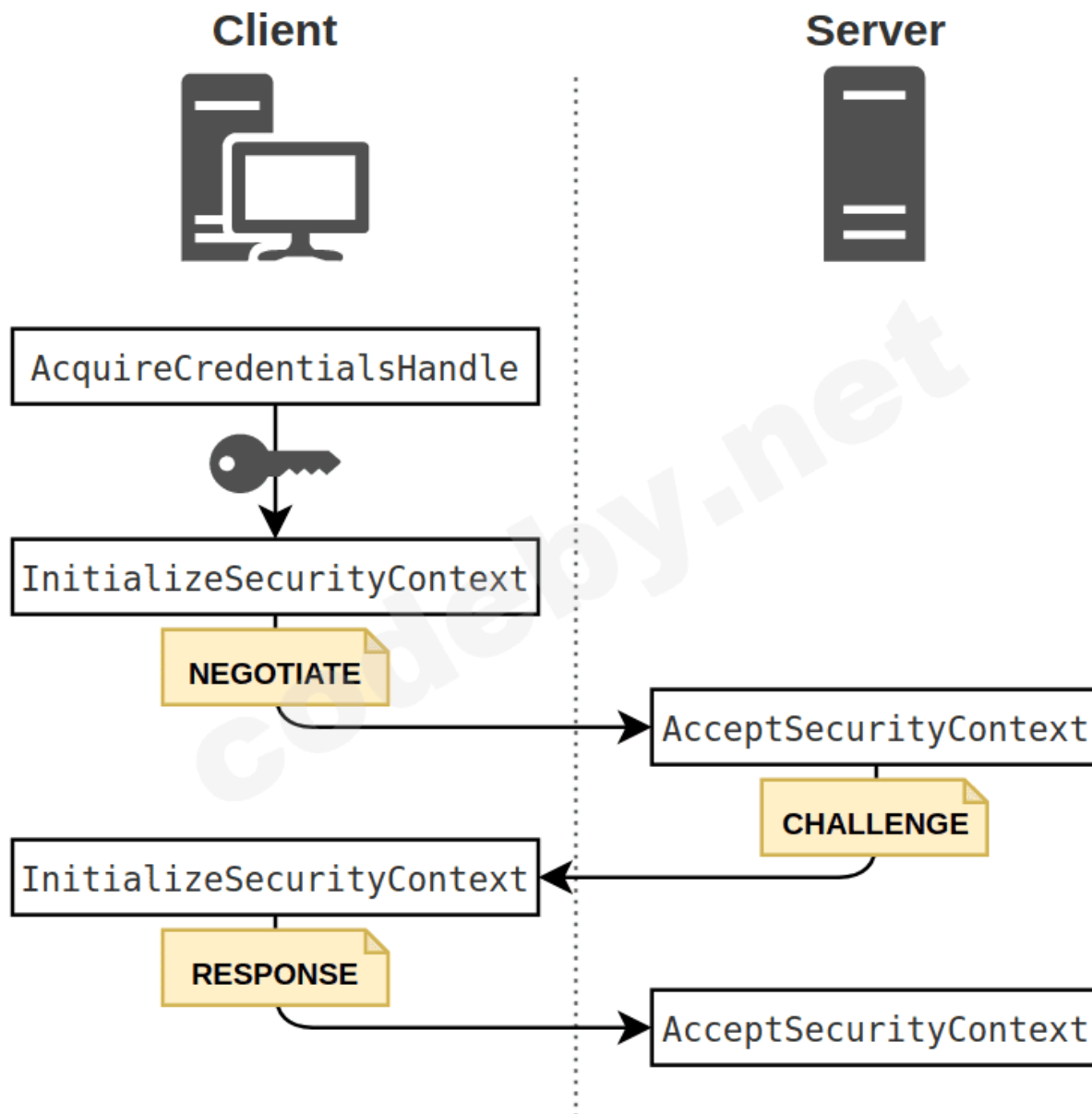
Интерфейс SSPI, или **Security Support Provider Interface**, является интерфейсом, предложенным корпорацией Майкрософт для стандартизации аутентификации, независимо от типа используемой аутентификации. К этому интерфейсу могут подключаться различные пакеты для работы с различными типами аутентификации.

В нашем случае нас интересует пакет NTLMSSP (**NTLM Security Support Provider**), но есть также пакет для аутентификации Kerberos, например.

Не вдаваясь в подробности, интерфейс SSPI предоставляет несколько функций, включая `AcquireCredentialsHandle`, `InitializeSecurityContext` и `AcceptSecurityContext`.

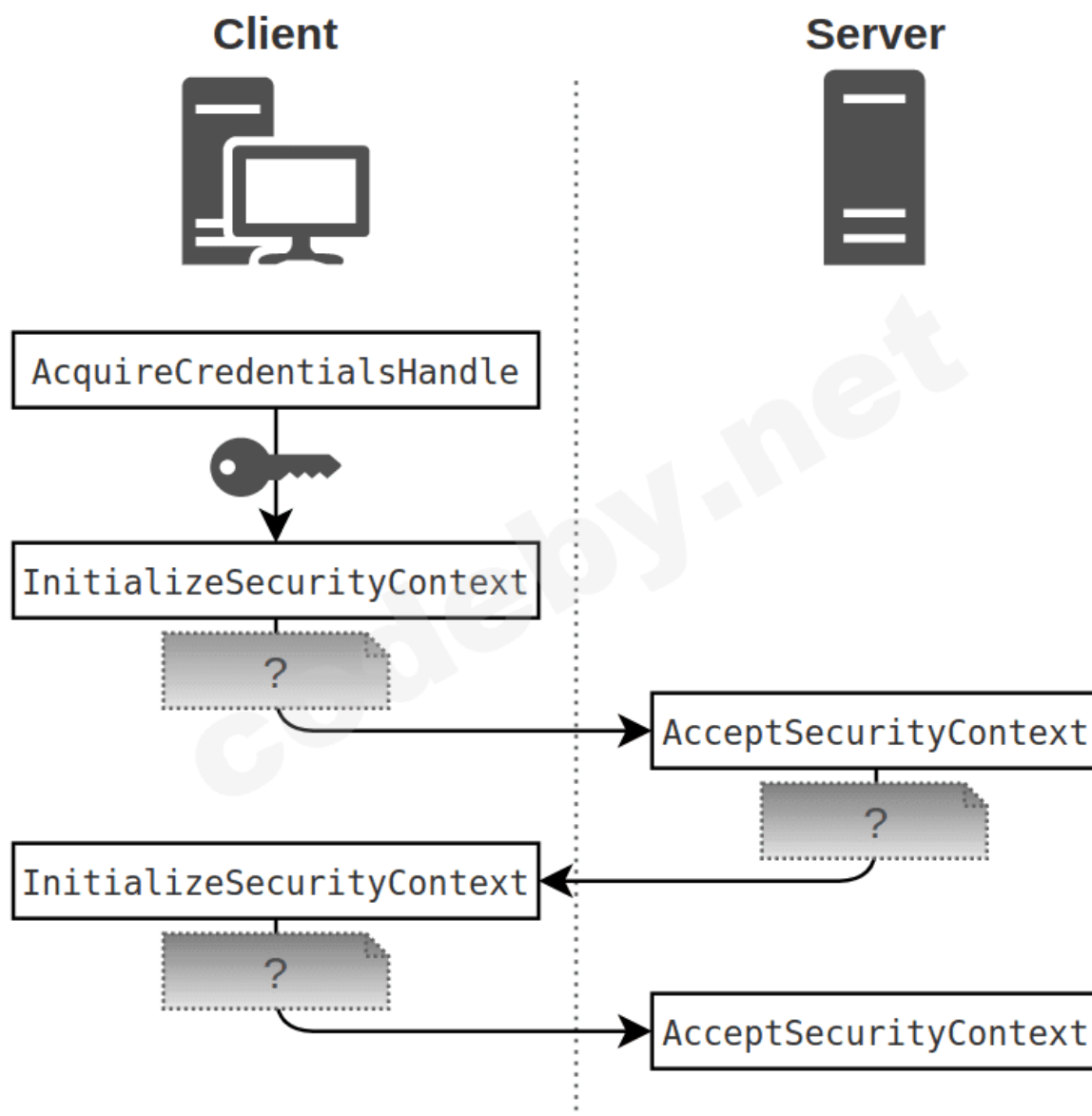
Во время NTLM-аутентификации и клиент, и сервер будут использовать эти функции. Шаги описаны здесь лишь вкратце.

1. Клиент вызывает `AcquireCredentialsHandle`, чтобы получить косвенный доступ к пользовательским учетным данным.
2. Затем клиент вызывает `InitializeSecurityContext`, функцию, которая при первом вызове создаст сообщение 1 типа, таким образом, типа **NEGOTIATE**. Мы знаем это, потому что нас интересует NTLM, но для программиста не имеет значения, что это за сообщение. Важно только отправить его на сервер.
3. При получении сообщения сервер вызывает функцию `AcceptSecurityContext`. Эта функция затем создаст сообщение 2 типа, **CHALLENGE**.
4. При получении этого сообщения клиент снова вызовет функцию `InitializeSecurityContext`, но на этот раз передаст **CHALLENGE** в качестве аргумента. Пакет NTLMSSP позаботится обо всем, чтобы вычислить ответ, зашифровав вызов, и выдаст последнее сообщение **AUTHENTICATE**.
5. После получения этого последнего сообщения сервер также снова вызывает `AcceptSecurityContext`, и проверка подлинности будет выполнена автоматически.



Причина, по которой эти шаги объясняются, заключается в том, чтобы показать, что в реальности, с точки зрения клиента или сервера, **3 структура сообщений, которыми обмениваются, не имеет значения**. Мы, зная протокол NTLM, понимаем, чему соответствуют эти сообщения, но и клиенту, и серверу все равно. Эти сообщения описаны в документации Microsoft как **ораче токены**.

Это означает, что эти 5 шагов полностью независимы от типа клиента или сервера. Они работают независимо от используемого протокола до тех пор, пока в протоколе есть что-то, что позволяет так или иначе обмениваться этими ораче токенами между клиентом и сервером.



Поэтому протоколы приспособились к тому, чтобы найти способ поместить NTLMSSP, Kerberos или другую структуру аутентификации в определенное поле, и если клиент или сервер видит, что в этом поле есть данные, они просто передают их в `InitializeSecurityContext` или `AcceptSecurityContext`.

Этот момент очень важен, поскольку он четко показывает, что прикладной уровень (HTTP, SMB, SQL, ...) полностью независим от уровня аутентификации (NTLM, Kerberos, ...). Поэтому меры безопасности необходимы как для уровня **аутентификации**, так и для **прикладного уровня**.

Для лучшего понимания мы рассмотрим два примера протоколов приложений **SMB** и **HTTP**. Довольно легко найти документацию по другим протоколам. Это всегда один и тот же принцип.

Интеграция с HTTP

Так выглядит основной HTTP-запрос.

Код:

```
GET /index.html HTTP/1.1
Host: beta.hackndo.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: fr
```

Обязательными элементами в этом примере являются HTTP-глагол (**GET**), путь к запрашиваемой странице (**/index.html**), версия протокола (**HTTP/1.1**) или заголовок Host (**Host: beta.hackndo.com**).

Но вполне возможно добавить и другие произвольные заголовки. В лучшем случае удаленный сервер знает, что эти заголовки будут присутствовать, и он будет знать, как с ними обращаться, а в худшем - будет их игнорировать. Это позволяет нам иметь такой же запрос с некоторой дополнительной информацией.

Код:

```
GET /index.html HTTP/1.1
Host: beta.hackndo.com
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: en
X-Name: pixis
Favorite-Food: Beer 'coz yes, beer is food
```

Именно эта функция используется для передачи NTLM сообщений от клиента к серверу. Было решено, что клиент посылает свои сообщения в заголовке, называемом Authorization, а сервер в заголовке, называемом WWW-Authenticate. Если клиент пытается получить доступ к веб-сайту, требующему аутентификации, сервер отвечает добавлением заголовка WWW-Authenticate, и выделяет различные механизмы аутентификации, которые он поддерживает. Для NTLM, он просто скажет NTLM.

Клиент, зная, что требуется NTLM-аутентификация, отправит первое сообщение в заголовке авторизации, закодированном в base 64, потому что сообщение содержит не только печатные символы. Сервер ответит вызовом в заголовке WWW-Authenticate, клиент вычислит ответ и отправит его в Authorization. Если авторизация прошла успешно, сервер обычно возвращает код возврата **200**, указывающий, что все прошло хорошо.

Код:

```
> GET /index.html HTTP/1.1
> Host: beta.hackndo.com
```

```
> User-Agent: Mozilla/5.0
> Accept: text/html
> Accept-Language: en

< HTTP/1.1 401 Unauthorized
< WWW-Authenticate: NTLM
< Content type: text/html
< Content-Length: 0

> GET /index.html HTTP/1.1
> Host: beta.hackndo.com
> User-Agent: Mozilla/5.0
> Accept: text/html
> Accept-Language: en
=> Authorization: NTLM <NEGOTIATE in base 64>

< HTTP/1.1 401 Unauthorized
=> WWW-Authenticate: NTLM <CHALLENGE in base 64>
< Content type: text/html
< Content-Length: 0

> GET /index.html HTTP/1.1
> Host: beta.hackndo.com
> User-Agent: Mozilla/5.0
> Accept: text/html
> Accept-Language: en
=> Authorization: NTLM <RESPONSE in base 64>

< HTTP/1.200 OKAY.
< WWW-Authenticate: NTLM
< Content type: text/html
< Content-Length: 0
< Connection: close
```

Пока TCP сессия открыта, аутентификация будет эффективной. Однако, как только сеанс закроется, на сервере больше не будет контекста безопасности клиента, и потребуется новая аутентификация. Это часто случается, и благодаря механизмам SSO (**Single Sign On**) от Microsoft, они часто прозрачны для пользователя.

Интеграция с SMB

Рассмотрим другой пример, часто встречающийся в сети компании. Это протокол SMB, используемый для доступа к общим сетевым ресурсам, но и не только.

Протокол SMB работает с помощью команд. Они [документируются компанией Microsoft](#), и их много. Например, есть SMB_COM_OPEN, SMB_COM_CLOSE или SMB_COM_READ, команды открыть, закрыть или прочитать файл.

В SMB также есть команда, посвященная настройке SMB-сессии, и эта команда - SMB_COM_SESSION_SETUP_ANDX. Содержанию NTLM-сообщений в этой команде [посвящены два поля](#).

- LM/LMv2 Authentication: OEMPassword
- NTLM/NTLMv2 authentication: UnicodePassword

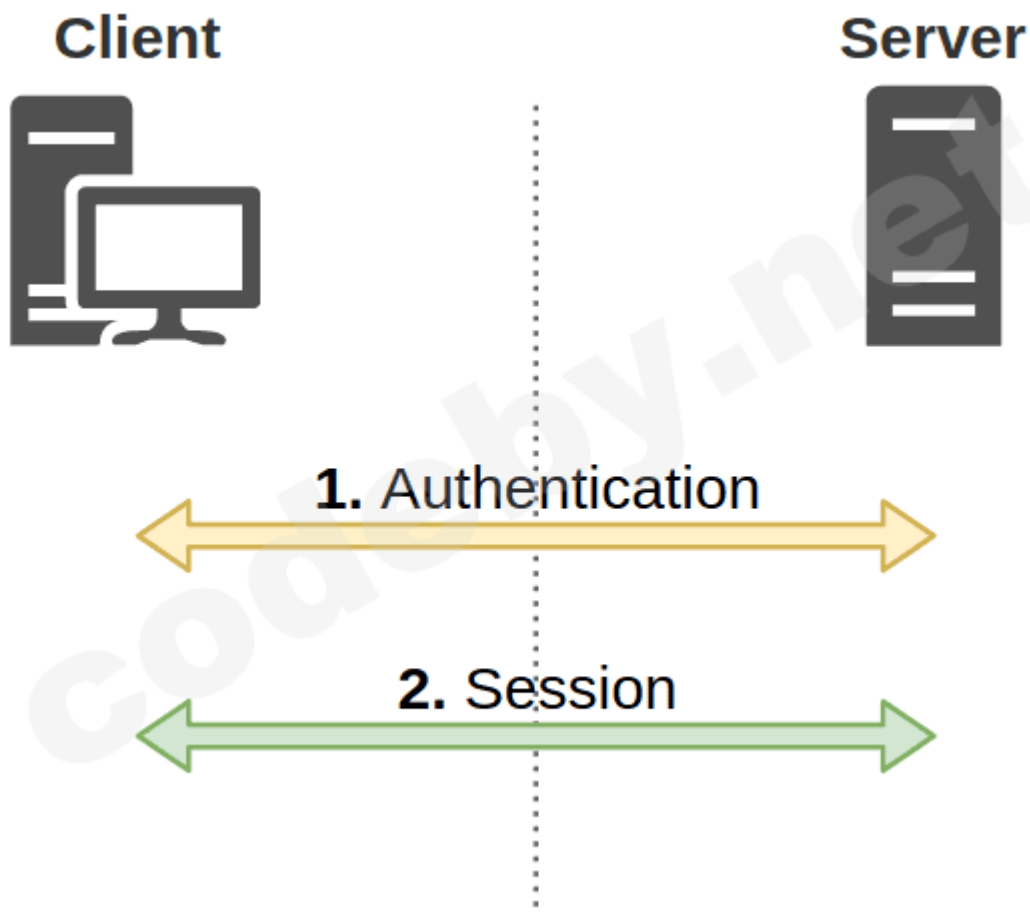
Важно помнить, что существует специальная SMB команда с выделенным полем для NTLM сообщений.

Вот пример SMB пакета, содержащего ответ сервера на аутентификацию.

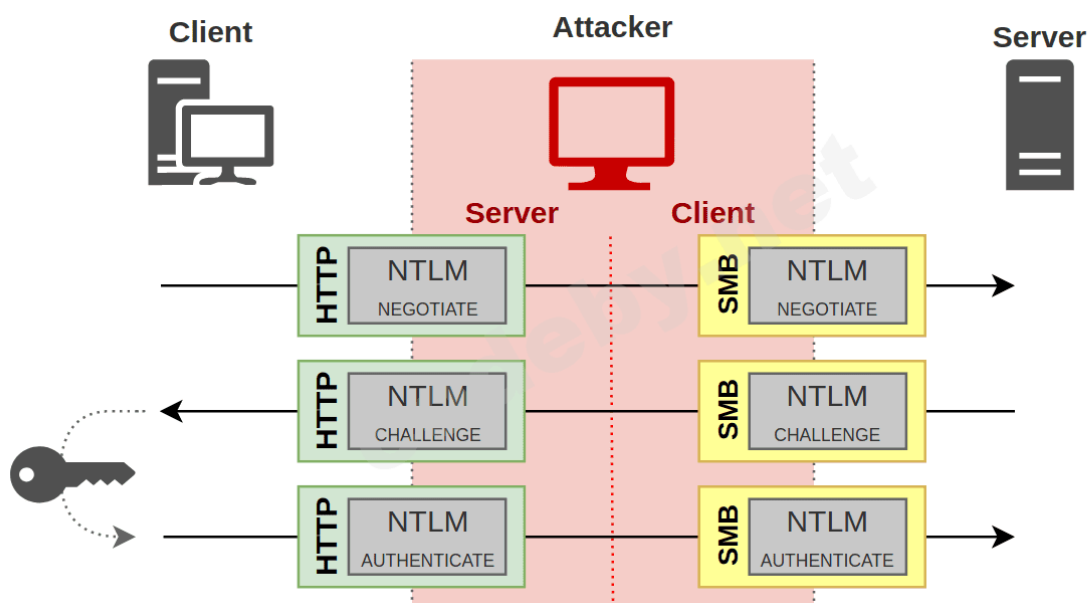
```
34 18.606498701 192.168.56.1 192.168.56.221 SMB 354 Session Setup AndX Response, NTLMSSP_CHALLENGE
> Frame 34: 354 bytes on wire (2832 bits), 354 bytes captured (2832 bits) on interface 0
> Ethernet II, Src: 0a:00:27:00:00:00 (0a:00:27:00:00:00), Dst: PcsCompu_37:55:f0 (08:00:27:37:55:f0)
> Internet Protocol Version 4, Src: 192.168.56.1, Dst: 192.168.56.221
> Transmission Control Protocol, Src Port: 445, Dst Port: 50415, Seq: 120, Ack: 302, Len: 300
> NetBIOS Session Service
> SMB (Server Message Block Protocol)
  > SMB Header
    > Session Setup AndX Response (0x73)
      Word Count (WCT): 4
      AndXCommand: No further commands (0xff)
      Reserved: 00
      AndXOffset: 0
    > Action: 0x0000
      Security Blob Length: 243
      Byte Count (BCC): 253
    > Security Blob: a181f03081eda0030a0101a10c060a2b0601040182370202...
      > GSS-API Generic Security Service Application Program Interface
        > Simple Protected Negotiation
          > negTokenTarg
            negResult: accept-incomplete (1)
            supportedMech: 1.3.6.1.4.1.311.2.2.10 (NTLMSSP - Microsoft NTLM Security Support Provider)
            responseToken: 4e544c4d53535000020000000a000a0038000000158289e2...
          > NTLM Secure Service Provider
            NTLMSSP identifier: NTLMSSP
            NTLM Message Type: NTLMSSP_CHALLENGE (0x00000002)
            > Target Name: ADSEC
              > Negotiate Flags: 0xe2898215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version,
                NTLM Server Challenge: 06515172c37197b0
              > Reserved: 0000000000000000
            > Target Info
              > Version 10.0 (Build 14393); NTLM Current Revision 15
```

Эти два примера показывают, что содержимое NTLM-сообщений не зависит от протокола. Оно может быть включено в любой протокол, который его поддерживает.

Тогда очень важно четко отличить аутентификационную часть, т.е. NTLM-обмены, от прикладной части, или сеансовой части, которая является продолжением обменов по протоколу, используемому после аутентификации клиента, например, просмотр вебсайта по HTTP или доступ к файлам на сетевом ресурсе, если мы используем SMB.



Поскольку эта информация является независимой, это означает, что человек-в-среде вполне может получить аутентификацию по HTTP, например, и передать ее на сервер, но с использованием SMB. Это называется **cross-protocol relay**.



Учитывая все эти аспекты, в следующих главах будут освещены различные слабые стороны, которые существуют или существовали, а также механизмы безопасности, которые вступают в действие для их устранения.

Часть 2

Последнее редактирование: